FoveaWAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation

Anonymous CVPR submission

Figure placeholder: concept figure, design pending

Planned content: one ConMoto frame overlaid with its real training labels; no generated imagery. The target at capture (solid) and when the commands take effect (translucent); the target's SE(3) poses at the 8 forecast horizons (axes); sparse 3D flow on the 14×14 patch points (arrows, near zero on the background). Task, layout, and styling to be decided.

Figure 1. **Concept figure (placeholder).** The caption will be written with the final figure.

Abstract

Manipulating moving objects requires actions that fit the scene when they execute, not when the image was captured, yet every millisecond spent inferring the scene's motion lets the target move further. World–action models forecast future images or latents at uniform fidelity, spending time on appearance while leaving the target's metric pose implicit; target-centric predictors are fast but ignore the surroundings that constrain or drive the motion. We observe that the parts of a dynamic scene need different precision: the target's pose decides contact to within millimeters, the bodies around it matter only through the clearance they leave and the motion they impart to the target, and the remaining background does not matter. We present FoveaWAM, a world–action model built on this role-aware 4D structure. It encodes the target's motion as a metric SE(3) pose history, so one RGB-D frame suffices as input. It learns to forecast the target's future SE(3) trajectory precisely and the whole scene coarsely as sparse 3D flow, with forecasts used only in training. Together, these choices let FoveaWAM act in [XX] ms on one RTX 5090, [N]× faster than video world–action baselines. We also introduce ConMoto, an open-source benchmark of 14 simulated dynamic manipulation tasks in which the scene keeps moving throughout each policy's inference. FoveaWAM achieves [XX.X]% success on ConMoto, [XX.X] points above the strongest baseline, and [XX]% across six real-robot tasks with diverse target motion.

1. Introduction

Learning-based manipulation has advanced rapidly, from diffusion policies to vision–language–action (VLA) and world–action models (WAMs) [5, 6, 12, 20, 42, 48, 62], yet most of this progress assumes that objects stay still unless the robot moves them. In factories, kitchens, and human–robot handovers, objects keep moving while the robot perceives, computes, and acts. In such scenes, an action must fit the scene at the moment it executes, not the moment its observation was captured, so a policy has to know how the scene moves. That knowledge must itself be inferred, and inference takes time during which the scene keeps moving: the longer a policy computes to anticipate the motion, the further ahead it must anticipate.

Recent dynamic policies add a memory of past motion, typically image-space cues such as optical flow [14, 27], and a forecast of the future [27, 38, 55]. Video-based WAMs forecast whole future frames or latents at uniform fidelity, spending much of their computation on visual detail [42, 48]. Compact future codes [28] reduce this cost and metric 3D scene dynamics [53] add geometry, but both still treat the whole scene uniformly. In each case, the quantity that decides contact, the target's metric pose at execution time, stays implicit. Target-centric predictors and dynamic VLAs with object-level forecasts sit at the other end [14, 17]: they are fast but anticipate only the target, leaving out the surroundings that can block or redirect its motion. Neither approach allocates precision according to what contact requires from each part of the scene. This raises a central question: what must a policy know about a moving scene, and how precisely, to make contact while keeping inference latency low?

Our key observation is that a dynamic policy needs to know very little precisely (Fig. 1). Contact demands millimeter precision in the target's pose at execution time; the surroundings matter only through the clearance they leave and the motion they impart to the target, so they need only be known coarsely. Building on this observation, we introduce FoveaWAM, a world–action model that learns a role-aware metric 4D representation, precise where contact

demands it (Fig. 2). To supply motion information without encoding past image frames, FoveaWAM pairs a single RGB-D frame with a short history of the target's metric SE(3) poses. A world stream learns the representation by forecasting the target's future SE(3) trajectory and the motion of the whole scene as a sparse 3D flow. An action stream reads the world stream's scene and summary tokens to generate action chunks, with explicit forecasting confined to training [52]. FoveaWAM thus combines precise anticipation of target motion with low inference latency, limiting how much the scene changes between observation and action.

We build ConMoto, an open-source benchmark of 14 simulated dynamic manipulation tasks in which the scene keeps moving throughout each policy's inference, so every method pays for its own latency. FoveaWAM achieves [XX.X]% success on ConMoto, [XX.X] points above the strongest baseline, at a p95 latency of [XX] ms, [N]× lower than that of video world–action baselines. It also leads on 24 DOMINO tasks [14] and succeeds in [XX]% of trials on six real-robot tasks, against [XX]% for the strongest baseline. Controlled ablations test the contributions of target-motion history and future supervision, and compare role-aware with uniform-fidelity forecasts to examine the allocation of precision.

Our contributions are threefold:

- We show that a dynamic policy needs millimeter precision only in the target's pose at execution time, and instantiate this principle as role-aware 4D modeling: a precise metric SE(3) trajectory for the target and a coarse sparse 3D flow for the whole scene.
- We develop FoveaWAM, a world–action model that combines metric target-pose histories with role-aware predictive supervision for low-latency action generation.
- We introduce ConMoto, an open-source dynamic manipulation benchmark that charges each policy its own inference latency, and demonstrate gains over strong baselines on ConMoto, 24 DOMINO tasks, and six real-robot tasks.

2. Related Work

Dynamic manipulation. Compensating for the time between sensing and acting is a classical problem, from control with dead time [35] to catching systems that track the target with Kalman-type filters and plan an interception [1–3, 30]; such systems are precise but engineered for each object and task. Learned policies have recently been brought to moving scenes [46]. PUMA reads historical optical flow and is supervised with future object features [14], and DynamicWAM conditions a WAM on flow and kinematic descriptors and executes it with real-time chunking [27]. Both infer past motion from image-space cues, whereas FoveaWAM takes the target's metric SE(3) pose history as input and observes only the current RGB-D frame. Other dynamic policies forecast salient tokens [38], future images [56], compact future codes [28], or 3D scene dynamics [53] and condition their actions on these forecasts at test time. DynaWM adapts a base VLA's action chunks for dynamic manipulation [10], and DynamicManip synthesizes dynamic demonstrations from a static one [22]. Execution schemes reduce or hide latency by overlapping inference with execution [33, 46], keeping consecutive chunks consistent [7, 25], or conditioning on the execution time [39]. They decide when actions are computed and applied, not what the policy represents. DOMINO [14] and PhysMani-Bench [53] provide diverse dynamic-manipulation tasks, but their released default evaluation pipelines advance the simulation only after a policy has returned its prediction. ConMoto advances the scene during inference and charges each method its measured latency.

Predictive supervision in WAMs. World–action models (WAMs) generate future video and actions in one network [21, 42, 48, 60], building on video prediction as a policy objective [40, 45]. Because generating video is costly, recent work changes what is forecast, such as dynamic regions, depth, and semantic features [55], flow and geometry alongside RGB [51], object masks or slots [26, 49], or compact latents [37, 59], and asks whether the forecast is needed at test time. Fast-WAM retains most of the benefit of video modeling without test-time imagination [52], and PUMA and SG-WAM drop their predictors after training [14, 57]. FoveaWAM keeps this training-only use but forecasts primarily a metric 4D state rather than pixels, features, or latents, so the target's future pose is supervised explicitly instead of being left implicit.

Object-centric and geometric motion representations. Metric 3D inputs make policies more precise and sample-efficient [32, 54]. Motion has been represented by 2D point tracks [4, 44, 47], 3D flow [15, 18, 50, 53], and object keypoints or SE(3) trajectories [17, 29, 36], while rigid-body decompositions have long been used to learn scene dynamics [8, 9]. These works describe the scene with one kind of representation, such as dense flow or tracks everywhere, or a pose for each object. Target-centric predictors, including dynamic VLAs with object-level forecasts, model only the target's future [14, 17] and leave out the bodies that block or push it. DOCIR separates the robot, the target, and obstacles, but uses these distinctions only as input encodings [13]. FoveaWAM assigns precision by role: a coarse sparse 3D flow describes the whole scene, while the target, on which contact depends to within millimeters, is additionally described by its SE(3) history and forecast.

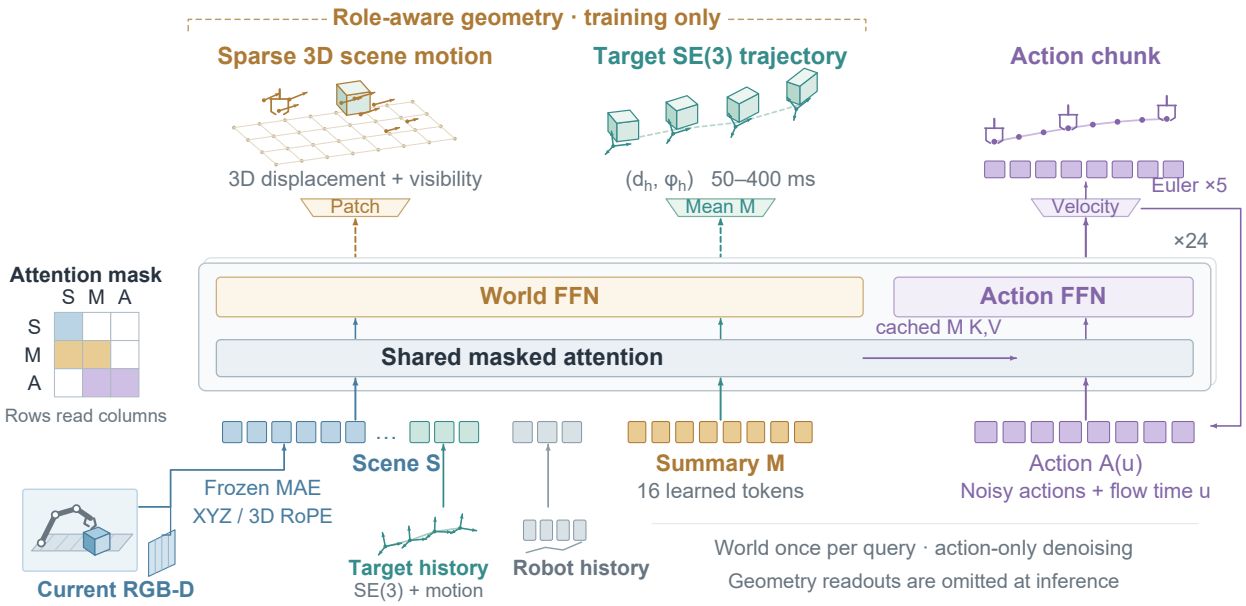


Figure 2. **FoveaWAM architecture.** Read from bottom to top. Current RGB-D and target/robot histories feed the world stream. World and action streams share attention and use separate FFNs. Training-only heads (dashed) predict sparse whole-scene 3D displacement and visibility from patch states, and target SE(3) trajectories from the mean summary. Actions read scene and summary tokens through the structured mask. At inference, scene and summary keys and values are computed once per query and reused across 5 Euler steps; forecast heads are skipped. Geometry drawings are schematic.

3. FoveaWAM

We formulate dynamic manipulation as generating time-indexed command chunks (Sec. 3.1) from a representation supervised, at the execution times of these commands, with the scene's sparse 3D flow and the target's SE(3) trajectory (Fig. 2). We describe the metric inputs (Sec. 3.2), this supervision (Sec. 3.3), and how an action stream reads the representation to generate the chunk (Sec. 3.4).

3.1. Problem Formulation

We study manipulation in scenes that keep moving while the policy computes. At query t, a policy π learned from demonstrations reads an RGB image I_t and a depth map D_t captured at time τ_t, together with histories $\mathcal{T}_t$ of the target pose $T(\tau) = (R(\tau), \mathbf{p}(\tau)) \in \mathrm{SE}(3)$ and $\mathcal{R}_t$ of the robot state, all in the robot base frame, and outputs a chunk of H commands

$$(\mathbf{a}_{t,0}, \ldots, \mathbf{a}_{t,H-1}) \sim \pi\big(\cdot \mid I_t, D_t, \mathcal{T}_t, \mathcal{R}_t\big), \qquad (1)$$

where command j is meant for time $\tau_t + j\Delta_a$, with control period Δ_a.

Each task specifies a *target*, whose pose the robot must match within a contact or containment tolerance. The surroundings that matter form the *context*: obstacles, which the robot must clear, and drivers, which move the target. The chunk becomes available only after inference, so each executed command must account for the target and the context at its execution time rather than at τ_t.

3.2. Metric Scene and Motion Inputs

FoveaWAM encodes the target's motion precisely and its context through the current geometry: the current RGB-D frame gives the metric layout of the scene, and a short history of target poses gives the motion that one frame does not show, which also reflects how drivers have moved the target.

Current scene. FoveaWAM anchors each patch token at a metric 3D point, so that the observation shares the base frame of the forecasts and the actions. A frozen MAE ViT-B/16 [16] turns the RGB image into 196 patch tokens on a 14×14 grid, and the depth map back-projects the center of patch i to a point $\mathbf{x}_i \in \mathbb{R}^3$ in the base frame. A linear embedding of $\mathbf{x}_i$ is added to the patch token, and $\mathbf{x}_i$ drives 3D rotary position embeddings of the patch queries and keys, so that attention between patches depends on their relative 3D positions. The points $\mathbf{x}_i$ are also the source points of the scene flow (Sec. 3.3).

Target motion. Instead of past frames, from which motion must be recovered by comparing images, FoveaWAM receives the target's recent motion as a metric SE(3) state. The history spans the last 350 ms in 8 samples at 20 Hz; its 8 tokens hold the current target pose and the seven motions

between consecutive samples, each with its translation, relative rotation, time interval, and finite-difference velocity and acceleration in physical units. The velocity and acceleration that decide where the target will be thus enter the model directly, at the cost of a few low-dimensional tokens; Sec. 4.3 removes this history to measure its effect. In simulation, the poses come from the simulator; on the robot, a causal tracker estimates them from the RGB-D stream.

World stream. The patch tokens, the target tokens, and 8 robot tokens with the end-effector states at the times of the target history form the 212 scene tokens. Together with 16 learnable summary tokens that aggregate them, they make up the world stream.

3.3. Role-Aware 4D Predictive Supervision

FoveaWAM supervises its representation with two geometric forecasts of different precision: the motion of the whole scene as a sparse 3D flow on the patch grid, and, because the target's pose decides contact, the full SE(3) trajectory of the target. Both are expressed in the base frame as changes from the current state at the times $\tau_t + \delta_h$, with $\delta_h = h{\cdot}50\,\text{ms}$ for $h = 1, \dots, 8$, from 50 ms to 400 ms after capture; together with the current state, these horizons cover the execution times of all commands in the chunk.

Scene flow. At the resolution of the patch grid, the flow covers every surface that the patches see: the robot, the target, and the context, whose clearance and motion the commands must account for. For each of the 196 points $\mathbf{x}_i$ and each horizon h, a linear head on the final state of patch token i predicts the point's base-frame displacement $\hat{F}_{ih} \in \mathbb{R}^3$ and a visibility logit $\hat{o}_{ih}$, trained with

$$\mathcal{L}_F = \frac{1}{|\mathcal{V}_F|} \sum_{(i,h)\in\mathcal{V}_F} \Big[\tfrac{1}{3} \big\| (\hat{F}_{ih} - F_{ih})/s \big\|^2 + \mathrm{BCE}(\hat{o}_{ih}, o_{ih}) \Big], \tag{2}$$

where F_{ih} is the displacement of the surface point at $\mathbf{x}_i$, o_{ih} its visibility, s a fixed scale fitted on the training labels, and $\mathcal{V}_F$ the set of valid labels.

Target trajectory. The flow samples the target only at the few patches it covers, too sparse to determine its rotation, so a dedicated linear head on the mean of the final summary states forecasts its rigid motion directly: for each horizon h, the target's displacement $\hat{\mathbf{d}}_h$ and a rotation increment $\hat{\boldsymbol{\phi}}_h$, trained with

$$\mathcal{L}_P = \frac{1}{|\mathcal{V}|} \sum_{h\in\mathcal{V}} \Big[\|\hat{\mathbf{d}}_h - \mathbf{d}_h\|^2 + \lambda_R \, d_{\mathrm{SO}(3)}\big(\hat{R}_h, R_h\big)^2 \Big], \tag{3}$$

where $\mathbf{d}_h = \mathbf{p}(\tau_t{+}\delta_h) - \mathbf{p}(\tau_t)$ and $R_h = R(\tau_t{+}\delta_h)$ are the true displacement and orientation, $\hat{R}_h = \mathrm{Exp}(\hat{\boldsymbol{\phi}}_h^{\wedge})R(\tau_t)$, $d_{\mathrm{SO}(3)}$ is the geodesic distance, and $\mathcal{V}$ is the set of horizons with valid labels. The rotation term matters for contact: a rotation error θ can displace a contact point at distance r from the target's origin by up to $2r\sin(\theta/2)$. For example, a 5° error at $r = 10\,\text{cm}$ can shift the contact point by nearly 9 mm.

3.4. World–Action Learning and Inference

Action stream. The action stream generates the chunk by conditional flow matching [23]. It holds $H = 8$ tokens, each embedding one noisy command: an end-effector translation increment, a rotation increment, and a gripper command. In every layer, one attention spans both streams with shared projections, followed by separate feed-forward networks; world tokens use layer normalization, and action tokens use adaptive layer normalization conditioned on the flow time. With S, M, and A denoting the scene, summary, and action tokens, the attention mask allows $S \to S$, $M \to \{S, M\}$, and $A \to \{S, M, A\}$, where $X \to \mathcal{Y}$ means that tokens in X attend to tokens in $\mathcal{Y}$. The world stream never reads the actions. The action stream reads, in every layer, both the patch states shaped by scene-flow supervision and the summary states shaped by target-trajectory supervision.

Training objective. For a chunk $\mathbf{a}_1$ normalized with training statistics, noise $\mathbf{a}_0 \sim \mathcal{N}(0, I)$, and flow time $u \sim \mathcal{U}[0, 1]$, the action stream v_ψ regresses the flow-matching velocity

$$\mathcal{L}_A = \mathbb{E}\,\mathrm{MSE}\big(v_\psi(\mathbf{a}_u, u \mid \mathcal{S}_t),\, \mathbf{a}_1 - \mathbf{a}_0\big), \tag{4}$$

where $\mathbf{a}_u = (1-u)\mathbf{a}_0 + u\mathbf{a}_1$, $\mathcal{S}_t$ denotes the inputs in Eq. (1), and the MSE averages over valid commands and action dimensions. The world and action streams are trained jointly with

$$\mathcal{L} = \mathcal{L}_A + \lambda_F \mathcal{L}_F + \lambda_P \mathcal{L}_P + \lambda_{\mathrm{aux}} \mathcal{L}_{\mathrm{aux}}, \tag{5}$$

where $\mathcal{L}_F$ and $\mathcal{L}_P$ are the flow and pose losses of Eqs. (2) and (3), and $\mathcal{L}_{\mathrm{aux}}$ sums auxiliary appearance losses.

Inference. At deployment, FoveaWAM skips the forecast heads, as recent world–action models do [21, 52, 55]. Because world tokens attend neither to the actions nor to the flow time, the world stream runs once per query, and the action stream integrates 5 Euler steps on the cached keys and values of the scene and summary tokens. Commands are indexed by time from capture, so the policy needs no latency input: the executor hands over each chunk at the first control-period boundary at or after inference completes and starts from the command indexed to that boundary. On the robot, the latency of a query also includes tracking the target

in the newest frame. Further implementation details are in the supplementary material.

4. Experiments

We test whether role-aware precision and low latency together improve manipulation when the scene keeps moving during inference. We first compare FoveaWAM with existing policies, each charged with its own latency (Sec. 4.2), then analyze which parts of its design account for the results (Sec. 4.3), and finally verify it on a real robot with tracked target poses (Sec. 4.4).

4.1. Experimental Setup

ConMoto. Unlike the evaluators of DOMINO [14] and PhysMani-Bench [53], which advance the simulation only after a policy has returned its prediction, ConMoto shares one physical clock among observation, scene motion, and action execution, so the scene keeps moving while a policy computes. It contains fourteen tasks in five families, built in MuJoCo/robosuite [41, 61]: grasping a moving target (harmonic, conveyor, translating, rotating, and occluded motion), placing into a moving receptacle, passing a wand along a rotating wire, capturing rolling or sliding objects, and grasping after an impact (Fig. 3). Experts with privileged simulator state provide 800 demonstrations per task. We select configurations on development scenes and report all ConMoto results on 1,400 held-out scenes (100 per task).

Evaluation. During each query, physics advances by the method's inference latency, drawn from its latency distribution measured in its native deployment setting at batch size 1 on one RTX 5090, the GPU of our robot. All methods share one executor, which executes each chunk from the command meant for the control-period boundary at which the chunk arrives. Besides this native setting, we add 150 ms of latency to every method and, as a reference without the cost of computation, pause physics during inference. We report the task-balanced success rate (SR) and the p95 time-to-action, the measured time from an observation to its first command; paired 95% intervals come from a bootstrap over held-out scenes. The supplementary material gives the full evaluation protocol.

Baselines. We compare with the reactive policies ACT [58] and Diffusion Policy [12] and the VLA $\pi_{0.5}$ [5]; the dynamic VLAs DynamicVLA [46] and PUMA [14]; the WAM OpenWAM-α [42]; and the dynamic WAM DynamicWAM [27] (Table 1). Each baseline is trained on the same demonstrations with its official architecture and a task-adapted recipe. FoveaWAM receives a single agent-view RGB-D frame and the target's SE(3) history, without past frames. To give baselines the same target-motion information, $\pi_{0.5}$ and DynamicWAM also run with our SE(3) history tokens (+SE(3)).

Table 1. **Results on ConMoto.** Task-balanced SR (%) on 1,400 held-out scenes with each method's native latency, with 150 ms added, and with physics paused during inference. p95: time-to-action in each method's native deployment setting on one RTX 5090. History: inputs that carry past motion beyond the current frame. OpenWAM-α uses its official caching and compilation, and (off) disables both. Best and second best per column are highlighted. [One-sentence takeaway, e.g., FoveaWAM attains the highest native SR at a fraction of the WAMs' latency.]

Method	History			SR (%) ↑			
	Wrist RGB	Flow	SE(3)	Native	+150 ms	Paused	p95 (ms) ↓
ACT [58]				41.6	24.3	40.6	**5.8**
DP [12]	✓			30.7	15.9	31.0	68.9
$\pi_{0.5}$ [5]				**53.8**	27.8	66.0	59.6
$\pi_{0.5}$+SE(3)			✓	52.8	**28.2**	**74.9**	61.2
DynamicVLA [46]	✓			50.9	23.4	54.6	97.3
PUMA [14]		✓		—	—	—	—
OpenWAM-α [42]				46.1	24.7	58.3	151.7
OpenWAM-α (off)				15.8	10.4	58.3	466.3
DynamicWAM [27]		✓		40.5	20.6	49.1	149.6
DynamicWAM+SE(3)		✓	✓	36.8	21.9	64.1	153.3
FoveaWAM			✓	—	—	—	—

Implementation. The transformer of FoveaWAM has 24 layers of width 1024 with 16 heads and 523.8M trainable parameters on top of the frozen MAE ViT-B/16 encoder. On ConMoto, it trains from random initialization with AdamW for at most 20 epochs, and we keep the checkpoint with the lowest validation loss.

4.2. Dynamic Manipulation Results

Comparison with baselines. With its native latency, FoveaWAM reaches [XX.X]% SR on held-out ConMoto scenes (Table 1), [X.X] points above the strongest baseline, [name], and [X.X] points above the strongest baseline with the same SE(3) history, [name]. Because the +SE(3) baselines receive the same target-motion input, the second margin measures what the representation of FoveaWAM adds beyond that input. Its p95 latency of [XX] ms is [X]× lower than that of the video WAMs, OpenWAM-α and DynamicWAM, and close to that of [the fastest reactive policy, to be fixed]. Its gains are largest on [task types].

Effect of inference delay. [Charging computation changes the ranking:] with physics paused during inference, FoveaWAM leads [name] by [X.X] points, and with native latency by [X.X] points, while the video WAMs fall from

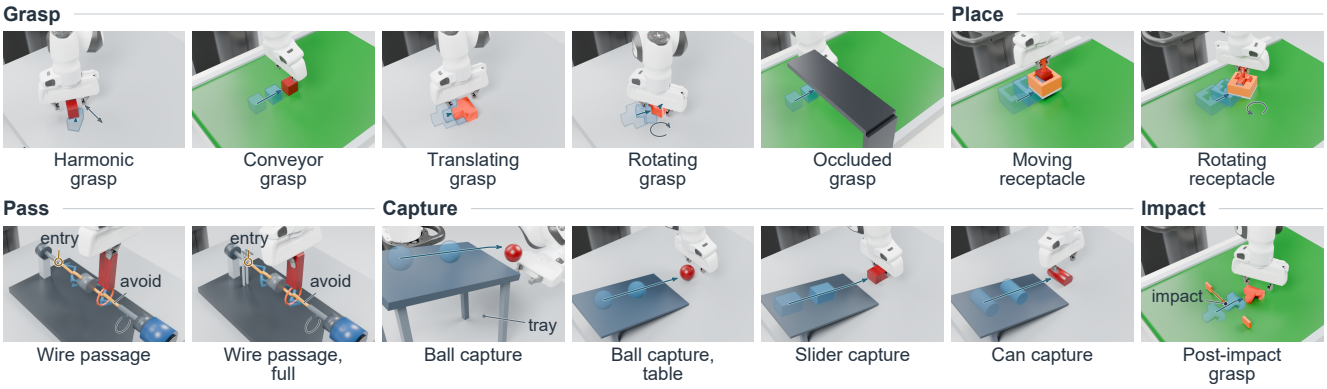


Figure 3. **ConMoto.** Fourteen tasks in five families, rendered from recorded simulator states with a common agent view. Translucent blue poses show selected history, fading with age; navy arrows follow recorded translational target motion. Local gray symbols denote rotation or oscillation qualitatively. An amber arrow shows the recorded striker approach; the *impact* label marks its contact location. Both Pass panels show passage: the wire is a target at entry and an obstacle along the path; Full additionally requires acquisition, exit and release. Occluded motion stays hidden. These native-expert examples illustrate task dynamics, not FoveaWAM outcomes; no future prediction is shown.

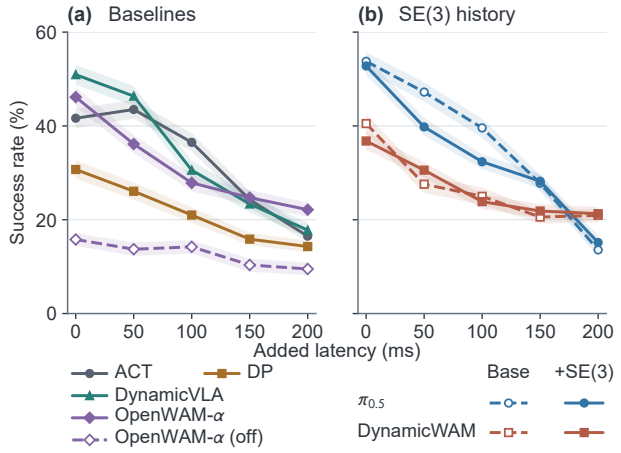


Figure 4. **Success under added latency.** Task-balanced SR on 1,400 held-out ConMoto scenes, one training seed. Delay is added to each method's native latency. (a) Baseline deployments; off disables OpenWAM-α caching and compilation. (b) Base versus +SE(3)-history variants. Bands: 95% task-stratified scene-bootstrap intervals. PUMA and FoveaWAM results pending.

[XX.X]% to [XX.X]% (Table 1). Adding 150 ms lengthens how far ahead every method must anticipate. FoveaWAM keeps [XX.X]% SR, whereas the reactive policies, which do not anticipate the target's motion, drop to [XX.X]% on average as their velocity error accumulates over the longer delay, and the video WAMs, already slow, drop to [XX.X]%. Fig. 4 extends this from 0 to 200 ms: [trend, e.g., the gap widens with every added step].

Results on DOMINO. To test the design beyond ConMoto, we train FoveaWAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of DOMINO [14] with its demonstrations and evaluate them with its native evaluator; as on ConMoto, FoveaWAM reads the target's simulator pose. Among the completed baselines, DynamicWAM has the highest overall SR at 29.3%, followed by PUMA at 18.3% and $\pi_{0.5}$ at 17.3% (Table 2); $\pi_{0.5}$ leads on Intercept tasks at 33.8%. [Add the accepted result and comparison for FoveaWAM.] More results are in the supplementary material.

Table 2. **DOMINO results** on 24 single-arm Franka clean-L1 tasks, with one training seed and the same 1,200 test scenes per method (50 per task). Intercept/Track contain 8/16 tasks; All weights all 24 tasks equally. SR and MS use a 0–100 scale and retain failures. Best and second-best reported values are highlighted; missing results are excluded.

Method	Intercept	Track	All ↑	MS ↑
$\pi_{0.5}$	**33.8**	9.1	17.3	31.1
DynamicWAM	18.3	**34.9**	**29.3**	**42.8**
PUMA	29.5	12.8	18.3	33.4
FoveaWAM	—	—	—	—

4.3. Analysis of Role-Aware Modeling

Five controls each change one part of FoveaWAM and keep its data and recipe (Fig. 5). Except for the control without target history, each changes only training losses whose heads are skipped at deployment, so it shares the inference of FoveaWAM. We report each change in SR with native latency and with 150 ms added.

Allocation of precision. Without the dedicated SE(3) supervision, the target receives only the uniform coarse scene-flow supervision shared by the whole scene, and SR drops by

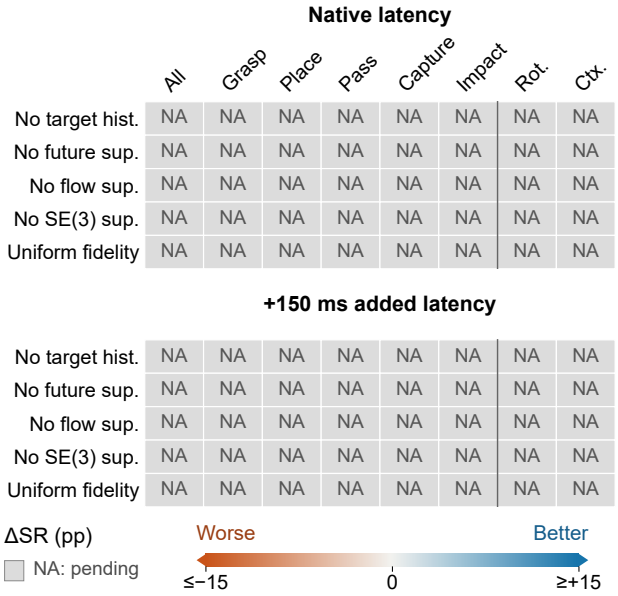


Figure 5. **Ablation effects by task (design preview; data pending).** ΔSR = SR(control) − SR(full), in percentage points; colors saturate at ± 15. Rows: the five controls of Sec. 4.3; *uniform coarse flow* removes the dedicated SE(3) supervision, so that the target receives only the scene-flow supervision shared by the whole scene. The full model reaches [XX.X]% SR with native latency and [XX.X]% with +150 ms. Rot./Ctx.: predeclared, overlapping rotation-sensitive/context-active groups (Table S3). Bold: the paired 95% scene-bootstrap interval excludes zero. NA: not available.

Table 3. **Real-robot results.** Success rate (%) on the six tasks of Sec. 4.4, 20 trials per task and method; All pools the trials. p95: end-to-end time-to-action (ms) on one RTX 5090, including tracking for FoveaWAM. Best per column is highlighted, including ties.

Method	T1	T2	T3	T4	T5	T6	All ↑	p95 (ms) ↓
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
ACT	—	—	—	—	—	—	—	—
FoveaWAM	—	—	—	—	—	—	—	—

[X.X] and [X.X] points. Without the scene-flow supervision, the target keeps its SE(3) trajectory while the rest of the scene, including the context, loses its motion supervision, and SR drops by [X.X] and [X.X] points. Both ablations keep all auxiliary objectives, as the full model does. If precision follows the roles, the SE(3) supervision should matter most on the predeclared rotation-sensitive tasks, and the flow supervision on the context-active tasks, whose context constrains the robot or moves the target. [Whether Fig. 5 shows this pattern, and what it implies for the allocation of precision.]

Target motion history. Removing the past target poses with their velocity and acceleration features, while keeping the current image, the current target pose, and the robot history, lowers SR by [X.X] and [X.X] points. [The drop grows with the added latency,] as expected if the history supplies the target's velocity: a longer delay lets a velocity error accumulate further. Because the history relies on pose estimates, we also perturb them; under synthetic correlated pose noise of [X mm and X°], FoveaWAM keeps [XX.X]% SR, [X.X] points below noise-free poses.

Future supervision and auxiliary objectives. Removing all future supervision, that is, the SE(3), scene-flow, and future appearance targets, while keeping the reconstruction of the current target, leaves inference unchanged yet lowers SR by [X.X] and [X.X] points, most on [tasks with non-uniform target motion]. Removing instead the auxiliary objectives $\mathcal{L}_{\text{aux}}$, including the current-target reconstruction, and keeping both geometric forecasts changes SR by [X.X] points, [what this implies for the role of the auxiliary objectives]. Offline, the learned SE(3) forecast is also more accurate than constant-velocity and constant-acceleration extrapolation of the same history ([X] vs. [X] and [X] mm at 400 ms). [One-sentence summary of which components matter, and where.]

4.4. Real-Robot Evaluation

Setup. We evaluate six tasks on a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera (Fig. 6): conveyor grasping (T1), placement into a moving box (T2), turntable grasping (T3), moving toy-car grasping (T4), tennis-ball grasping (T5), and block grasping from a ramp cart (T6). Baselines are the strongest simulation baseline, $\pi_{0.5}$; a low-latency imitation policy, ACT; and a dynamic WAM, DynamicWAM. All methods train or fine-tune on the same 100 teleoperated demonstrations per task, FoveaWAM from a 48-layer base pretrained on DROID [19], and run 20 trials per task with shared trial keys. Baselines receive RGB [and a wrist camera if their recipe uses one]; FoveaWAM receives the third-person RGB-D frame and FoundationPose [43] target estimates, updated at every control-period boundary before its query.

Results. FoveaWAM succeeds in [XX.X]% of all trials, against [XX.X]% for [the strongest baseline], a paired difference of [X.X] points (95% interval [[X.X, X.X]]; Table 3). Its p95 latency, including tracking, is [XX] ms, [within one control period], whereas [name] needs [XX] ms. The margin is largest on [tasks]. [Whether the ranking of Table 1 carries over to real perception, and what tracking costs.] [One trajectory-verified contact example: how the target's pose, rotation, or context explains a success of FoveaWAM or a failure of a baseline.] The dominant failure of FoveaWAM is

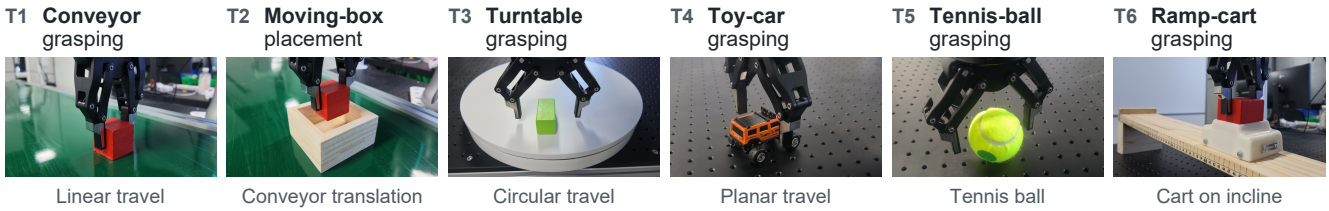


Figure 6. **Real-robot task setups.** Photographs of conveyor grasping (T1), placement into a moving box (T2), turntable grasping (T3), moving toy-car grasping (T4), tennis-ball grasping (T5), and block grasping from a ramp cart (T6).

[failure mode]. Additional details are in the supplementary material.

5. Conclusion

FoveaWAM spends precision where contact depends on it: a metric SE(3) history and forecast for the target and a sparse 3D flow for the whole scene, with the forecasts used only as training supervision. When every method is charged its own latency, these choices let FoveaWAM act both precisely and quickly: it outperforms reactive policies, VLAs, and world–action models, including ones built for moving targets, on ConMoto and on six real-robot tasks at a p95 latency of [XX] ms on ConMoto, and it also leads on DOMINO. [How charging latency changes the ranking relative to paused evaluation.] [The one or two strongest ablation findings.] We will release ConMoto so that future dynamic policies are evaluated with their latency charged.

Limitations. FoveaWAM requires a specified rigid target with reliable pose estimates, and its forecast ignores how the robot's own contact changes the target's motion; learning contact-conditioned target dynamics is a natural extension. Its target-trajectory head predicts a single future per horizon and so cannot represent multimodal motion; an uncertainty-aware forecast could address this. Our tasks contain no context that moves independently of the target; such scenes would call for a motion history of the context as well, for example of its scene flow.

LLM Usage Statement

We used OpenAI Codex to assist with literature review, code implementation and debugging, experiment execution, and language editing of the manuscript. Codex also assisted in developing plotting code and refining figure and manuscript layouts. The authors reviewed AI-assisted content, checked references against original sources, and validated the implementation through testing and experiments. The authors take full responsibility for the final manuscript, experimental results, and conclusions.

References

[1] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 2

[2] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Wiley, 2001.

[3] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 2

[4] Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision*, 2024. 2

[5] Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40, 2025. 1, 5, 15

[6] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, 2025. 1

[7] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, pages 33383–33407, 2025. 2, 14

[8] Arunkumar Byravan and Dieter Fox. SE3-Nets: Learning rigid body motion using deep neural networks. In *IEEE International Conference on Robotics and Automation*, pages 173–180, 2017. 2

[9] Arunkumar Byravan, Felix Leeb, Franziska Meier, and Dieter Fox. SE3-Pose-Nets: Structured deep dynamics models for visuomotor control. In *IEEE International Conference on Robotics and Automation*, 2018. 2

[10] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 2

[11] Ho Kei Cheng, Seoung Wug Oh, Brian Price, Joon-Young Lee, and Alexander Schwing. Putting the object back into video object segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 19

[12] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 5, 15

[13] David Emukpere, Romain Deffayet, Bingbing Wu, Romain Brégier, Michael Niemaz, Jean-Luc Meunier, Denys Proux, Jean-Michel Renders, and Seungsu Kim. Disentangled object-centric image representation for robotic manipulation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 15873–15879, 2025. 2

[14] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. In *European Conference on Computer Vision*, pages 314–333, 2026. 1, 2, 5, 6, 15, 17

[15] Jun Guo, Xiaojian Ma, Yikai Wang, Min Yang, Huaping Liu, and Qing Li. FlowDreamer: A RGB-D world model with flow-based motion representations for robot manipulation. *IEEE Robotics and Automation Letters*, 11(3):2466–2473, 2026. 2

[16] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 3, 12

[17] Cheng-Chun Hsu, Bowen Wen, Jie Xu, Yashraj Narang, Xiaolong Wang, Yuke Zhu, Joydeep Biswas, and Stan Birchfield. SPOT: SE(3) pose trajectory diffusion for object-centric manipulation. In *IEEE International Conference on Robotics and Automation*, 2025. 1, 2

[18] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20765–20779, 2026. 2, 12

[19] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 7, 12

[20] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, pages 2679–2713, 2025. 1

[21] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. 2, 4

[22] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 2

[23] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 4

[24] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang,

Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 19

[25] Yuejiang Liu, Jubayer Hamid, Annie Xie, Yoonho Lee, Max Du, and Chelsea Finn. Bidirectional decoding: Improving action chunking via guided test-time sampling. In *International Conference on Learning Representations*, pages 4594–4627, 2025. 2

[26] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026. 2

[27] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 1, 2, 5, 15

[28] Siyuan Ma, Yutian Zhang, Boshi Zhang, Qinglian Wu, Jiaqi Zhai, Dong Wei, and Xiaojin Huang. ForeTime-VLA: Causal future-token distillation from a world action model for conveyor-belt manipulation. *arXiv preprint arXiv:2608.20735*, 2026. 1, 2

[29] Lucas Manuelli, Wei Gao, Peter Florence, and Russ Tedrake. kPAM: Keypoint affordances for category-level robotic manipulation. In *Robotics Research: The 19th International Symposium ISRR*, pages 132–157. Springer, 2022. 2

[30] Naresh Marturi, Marek Kopicki, Alireza Rastegarpanah, et al. Dynamic grasp and trajectory planning for moving objects. *Autonomous Robots*, 43(5):1241–1256, 2019. 2

[31] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, pages 28085–28128, 2025. 19

[32] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, pages 785–799, 2023. 2

[33] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[34] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 12

[35] Otto J. M. Smith. Closer control of loops with dead time. *Chemical Engineering Progress*, 53(5):217–219, 1957. 2

[36] Rustin Soraki, Homanga Bharadhwaj, Ali Farhadi, and Roozbeh Mottaghi. ObjectForesight: Predicting future 3D object trajectories from human videos. In *European Conference on Computer Vision*, pages 437–453, 2026. 2

[37] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 115917–115933, 2026. 2

[38] Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic VLA manipulation. In *Conference on Robot Learning*, 2026. Accepted for publication. 1, 2

[39] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025. 2

[40] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, pages 92033–92052, 2025. 2

[41] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 5, 13

[42] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 1, 2, 5, 15

[43] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 7, 19

[44] Chuan Wen, Xingyu Lin, John Ian Reyes So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[45] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, pages 10641–10662, 2024. 2

[46] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 5, 15

[47] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Conference on Robot Learning*, pages 2475–2499, 2025. 2

[48] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 2

[49] Hanyang Yu, Haitao Lin, Jingbo Zhang, Wenyao Zhang, Chenghao Gu, Heng Li, and Ping Tan. MaskWAM: Unifying mask prompting and prediction for world-action models. *arXiv preprint arXiv:2606.13515*, 2026. 2

[50] Chengbo Yuan, Chuan Wen, Tong Zhang, and Yang Gao. General flow as foundation affordance for scalable robot learning. In *Conference on Robot Learning*, pages 1541–1566, 2025. 2

[51] Shanglin Yuan, Weiheng Zhao, Xin Shi, Haoyi Jiang, Xianda Guo, Liu Liu, Wenyu Liu, Wei Sui, and Xinggang Wang. DreamWAM: Beyond RGB future prediction for world action models. *arXiv preprint arXiv:2608.04996*, 2026. 2

[52] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 4

[53] Peng Yun, Shouwang Huang, Hao Li, Jinxi Li, Jianan Wang, and Bo Yang. PhysMani: Physics-principled 3D world model for dynamic object manipulation. In *European Conference on Computer Vision*, pages 20–39, 2026. 1, 2, 5

[54] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[55] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, XinQiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 1, 2, 4

[56] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE Robotics and Automation Letters*, 10(6):5209–5216, 2025. 2

[57] Ruiteng Zhao, Zhengshen Zhang, Yue Su, Wenshuo Wang, Jiahui Li, Zhiyuan Yang, Francis E. H. Tay, Marcelo H. Ang, and Haiyue Zhu. SG-WAM: Self-guided world modeling in geometry-aware policy space. *arXiv preprint arXiv:2608.01397*, 2026. 2

[58] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 5, 15, 19

[59] Ruijie Zheng, Jing Wang, Scott Reed, et al. FLARE: Robot learning with implicit world modeling. In *Proceedings of The 9th Conference on Robot Learning*, pages 3952–3971. PMLR, 2025. 2

[60] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[61] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 5, 13

[62] Brianna Zitkovich, Tianhe Yu, Sichun Xu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183, 2023. 1

A. Method and Training Details

A.1. Architecture

Table S1 lists the configuration of every module, in the order of the data flow described below.

Table S1. Architecture of FoveaWAM. Heads are skipped at deployment.

Module	Configuration
Visual encoder	frozen MAE ViT-B/16, 224×224 input, 196 patch tokens
Target-state tokenizer	$28 \to 1024 \to 1024$ GELU MLP, additive relative-time embedding
Robot-state tokenizer	linear, $7 \to 1024$
Tokens	212 scene (196 patch, 8 target-state, 8 robot), 16 summary, 8 action
Backbone	24 layers, width 1024, 16 heads, shared attention, SiLU FFN 4096 per stream, 504.0M parameters
Pose head	linear, mean summary state $\to 8 \times 6$
Flow head	linear per patch token, $8 \times (3{+}1)$
Appearance heads	linear per patch token: depth and moving-surface RGB as 16×16 pixels, 1024-d DINOv3 ViT-L/16 features
Target reconstruction	$1024 \to 256 \to 3072$ GELU MLP with sigmoid, 32×32 RGB
Action stream	flow matching, adaptive layer norm on the flow time, 5 Euler steps

Scene tokens. Each patch token of the frozen MAE ViT-B/16 [16] adds a learned view embedding and linear embeddings of its mean valid metric depth and of the base-frame position $\mathbf{x}_i$ of its back-projected center pixel; a patch without valid depth at its center receives no position embedding. Only the patch queries and keys receive 3D rotary embeddings from $\mathbf{x}_i$, and the same points are the source points of the sparse 3D flow. Past images never enter the model.

State tokens. The 8 target-state tokens sample the offsets $-350, -300, \ldots, 0$ ms. Each is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$: slots 0–6 describe the seven intervals between consecutive offsets, with velocities and accelerations from finite differences over the elapsed times, and slot 7 holds the current pose in the base frame. Each dimension is standardized with training statistics, invalid dimensions are zeroed, and three validity flags (pose, interval, acceleration) are appended; a slot without any valid feature is masked out of attention. The robot tokens hold the end-effector position, rotation vector, and gripper opening at the same offsets, and 16 learned embeddings form the summary tokens. The policy has no language input.

Backbone and heads. The mask of Sec. 3.4 sets blocked attention logits to $-\infty$ in every layer and head, in training and at deployment, so the world states are computed once per query and reused across the Euler steps. All heads read the final world states and are skipped at deployment; no token reads their outputs. The pose head reads the mean summary state ($\mathcal{L}_P$, Eq. 3). The flow head gives each patch point's displacement and visibility logit at every horizon ($\mathcal{L}_F$, Eq. 2, with scale $s = 0.7154605$ m and visibility weight 1); valid but invisible points still supervise their displacement. $\mathcal{L}_{\text{aux}}$ sums four losses: squared errors on future metric depth (valid pixels), on the RGB of surfaces that move by more than 1 mm, and on layer-normalized DINOv3 features [34], and an absolute error on the current target's RGB region reconstructed from the mean summary state.

Action stream. Each action token embeds a seven-dimensional command: a base-frame translation increment, a left-relative rotation vector, and a gripper command, relative to the measured end-effector pose at application. At deployment, the decoded commands are clipped to 0.10 m per translation axis and 0.5 rad per rotation axis, and the gripper changes state only when its command leaves a deadband of 0.5 in the ball-capture tasks and 0.1 in the others.

A.2. Training on ConMoto

Each control step of a demonstration anchors a window of 8 query positions, each with its own causal history and no attention across positions, giving 1,324,374 windows from the 11,200 demonstrations; 350 further episodes form the validation split. Flow labels follow the rendered surface point at each patch center through the ground-truth rigid motion of its MuJoCo geometry; a future point is visible if it projects into the image onto the same geometry within a depth tolerance of $\max(1\,\text{mm}, 0.001z)$. Command j of query t is labeled with the demonstration command issued at $\tau_t + j\Delta_a$, since d_{task} applies only at execution (Eq. (S1)), and commands are standardized over the training split. Each query position averages the action loss over 8 noise–time pairs, and each loss is normalized by its number of valid labels in the global batch. Table S2 lists the recipe.

A.3. Pretraining on DROID and Fine-Tuning

The real-robot base trains on DROID [19] (15 Hz) with the first exterior camera and its depth, using the 3D point tracks and calibrations released by PointWorld [18], matched to the raw episodes by robot states, commands, and camera identity. The 42,087 usable episodes yield 598,889 anchor windows (538,740 for training, 60,149 for validation); each anchor expands into 8 query positions, and only the anchor carries flow and DINOv3 labels. All action slots with an issued command are kept, without a speed filter. Each patch follows one PointWorld track that starts inside it, preferring

Table S2. Training hyperparameters on ConMoto.

Item	Value
Optimizer	AdamW, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 10^{-4}
Learning rate	2×10^{-4}, 5,174 warm-up updates from 0, cosine decay to 0
Batch, schedule	global batch 512 windows, at most 20 epochs, lowest-validation-loss checkpoint
Loss weights	$\lambda_F = \lambda_P = \lambda_R = 0.1$, $\lambda_{\text{aux}} = 0.1$ per target
Action noise	8 noise–time pairs per query position
Precision	trainable weights FP32, frozen encoder BF16, gradient clipping at 0.1
Inference	FP32, world states cached within a query
Hardware	[N] RTX PRO 6000 GPUs, [X] hours per run

the one with the largest consistent motion so that small moving objects are covered; 47.05% of the point–horizon slots carry a label.

DROID provides no object poses, so the base has no target-state tokens or pose head; it reads the current RGB-D frame and the robot history and trains the action loss and the sparse 3D flow with weight 1 and the future DINOv3 features with weight 0.1, without the visibility loss. Since DROID objects mostly move only when the robot moves them, the base mainly learns metric geometry and self-induced motion; exogenous target motion enters during fine-tuning. The base has 48 layers at width 1024 (1.02B trainable parameters) and trains for 20 epochs (21,060 updates) at a global batch of 512 on 64 GPUs with the optimizer and precision of Table S2. Each real-robot policy is fine-tuned from it on its task's demonstrations [updates, learning rate, and batch size], with the target-state tokens and missing heads freshly initialized; no real-robot policy uses simulation data or weights.

B. ConMoto and Evaluation Protocol

B.1. Benchmark Design

ConMoto comprises fourteen tasks in five families (Fig. 3) that cover translation, rotation, temporary occlusion, constrained passage, and abrupt changes of the target's motion. We will release the tasks, scenarios, success checks, and demonstrations.

Environment. The tasks run in MuJoCo 3.11.0 and robosuite 1.5.2 [41, 61] with a Franka Panda arm and a parallel gripper under robosuite's operational-space pose controller, with a control period of 50 ms and a physics step of 2 ms. A fixed third-person camera at $(0.68, -0.58, 1.62)$ m with a 40° vertical field of view renders 224×224 RGB-D images. An action is an end-effector translation and rotation-vector increment in the base frame with a gripper command, clipped to 0.10 m per translation axis and 0.5 rad per rotation component.

Success and termination. Table S3 gives the success condition of each task, checked at every physics step; an interrupted continuous condition resets its counter. An episode ends at success, at a failure such as unsafe contact, leaving the workspace, or a drop, or at the time limit.

Roles and task groups. Table S3 names the target of each task. A moving receptacle is the target because its pose decides containment; in the passage tasks, the wire entry, which the wand must match within 30 mm, is the target, and the rest of the wire is context that must not be touched. The two task groups of Fig. 5 were fixed before the ablations: a task is rotation-sensitive if its target rotates during the lead and its orientation matters at contact, which excludes the balls, and context-active if it has an obstacle clearance m or a driver (a conveyor, a ramp, or a striker).

B.2. Scenarios, Demonstrations, and Splits

Each scene samples the target's initial state and motion parameters uniformly within task-specific ranges: besides the speeds of Table S3, harmonic amplitudes of up to 35 mm at 0.25–0.7 Hz, yaw rates of up to 1.2 rad/s, wire rotation rates of 2.5–7.5°/s, and ramp slopes of 5–15°. An occluded scene shows the target for at least eight initial frames, hides it for 0.5–3.0 s, and shows it again for at least three frames. Only physically feasible scenes are kept, and evaluation replays the stored scenes.

The expert is a task-specific closed-loop state machine with privileged simulator state: the target's pose and velocity, the robot state, the task geometry, and true contact and stage events. It extrapolates the target and then tracks, descends, closes, and lifts in the grasping tasks, predicts the interception point in the capture tasks, and compensates the wire's phase in the passage tasks; its commands are clipped to 0.05 m and 0.5 rad per axis. A demonstration is kept if the expert succeeds, a replay of its actions reproduces the episode, and its path and jerk pass quality checks. The training pool holds 11,200 demonstrations, 800 per task, recorded at 20 Hz with RGB-D, robot states, commands, and simulator ground truth; policies receive only the inputs of Sec. 3.

We select configurations on 1,400 development scenes (100 per task) and report all ConMoto results on 1,400 held-out scenes, generated with the same ranges from seeds used in no training, validation, or development episode. Each task's held-out set holds its first 100 feasible scenes in seed order and is shared by all methods.

B.3. Real-Time Evaluation

Executor. All methods share one executor on the 50 ms control clock. Command j of query t is meant for $\tau_t + j\Delta_a$,

Table S3. **The fourteen ConMoto tasks.** Target: the body whose pose the policy receives (Sec. 3.1). Speed: target speed range (capture tasks: at the ramp exit). Every task requires its stages in order without a terminal failure; a grasp requires bilateral fingerpad contact, a lift of at least 0.05 m, and a 0.5 s hold. Limit: episode time limit. Groups of Fig. 5: R, rotation-sensitive; C, context-active (Appendix B.1). The released task configurations contain the exact thresholds and scenario ranges.

Task (code)	Target	Speed (m/s)	Success	Limit (s)	Group
Harmonic grasp (grasp)	block	0.03–0.20	Grasp the block moving harmonically and yawing, lift, and hold.	8	R
Conveyor grasp (t1_belt)	part	0.04–0.10	Grasp the part on the conveyor, lift, and hold.	8	C
Translating grasp (t1_xyz)	part	0.04–0.15	Grasp the translating part, lift, and hold.	8	–
Rotating grasp (t1_so3)	part	0.04–0.15	Grasp the translating and rotating part ($\|\omega\| \leq 1.2$ rad/s), lift, and hold.	8	R
Occluded grasp (t1_occ)	part	0.04–0.10	Grasp the part on the conveyor, which a canopy hides for 0.5–3.0 s, without unsafe contact with the canopy; lift and hold.	8	C
Moving receptacle (t2_trans)	receptacle	0.03–0.06	Place the held object fully inside the moving receptacle and release it to rest.	10	–
Rotating receptacle (t2_se3)	T-shaped receptacle	0.03–0.06	Place the held T-shaped part fully inside the translating and rotating T-shaped receptacle and release it to rest.	10	R
Wire passage (t3_phase)	wire entry	[X]	Align the held wand's loop with the rotating wire within 30 mm, then pass it along the wire to the end without contact.	21	R, C
Full wire passage (t3_full)	wire entry	[X]	Grasp the wand, complete the passage, exit the wire, and release the wand.	32	R, C
Ball capture (t4_sphere)	ball	[0.4–0.7]	Catch the ball after it leaves the ramp and before it touches the tray, lift, and hold.	6	C
Ball capture on table (t4_sphere_tabletop)	ball	[0.4–0.7]	Grasp the ball while it still rolls on the table after the ramp, lift, and hold.	6	C
Slider capture (t4_slider)	slider	[0.4–0.8]	Grasp the block while it still slides on the table after the ramp, lift, and hold.	6	C
Can capture (t4_can)	can	[0.4–0.7]	Grasp the can while it still rolls on the table after the ramp, before it tips over, lift, and hold.	6	R, C
Post-impact grasp (t5_replan)	part	0.12–0.14	After a striker deflects the part on the conveyor, grasp it along its grasp axis, lift, and hold.	6	R, C

and its execution time (Sec. 3.1) is

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + j\Delta_a + d_{\mathrm{task}}, \tag{S1}$$

where $d_{\mathrm{task}} = 50$ ms for the post-impact grasp, which applies each command 50 ms after it is issued, and $d_{\mathrm{task}} = 0$ otherwise. As in RTC [7], latency counts in whole control periods: a query with total latency L_t hands over its chunk at the first boundary at or after it ends, $k_t = \lceil L_t/\Delta_a \rceil$ periods after capture, and execution continues from command k_t, the one meant for that boundary. The policy then observes again and starts the next query; meanwhile the previous chunk continues, so a policy replans as often as its latency allows. With $L_t = 0$, the next query starts one period after capture. A chunk holds the method's native number H_m of commands (Table S4) and is dropped if $k_t \geq H_m$. Before the first chunk arrives and whenever a chunk runs out, the controller holds the current arm target and gripper command; no command is repeated. Every command of FoveaWAM executes within $(H-1)\Delta_a + d_{\mathrm{task}}$ of capture, inside its forecast horizons.

Protocols. In real time, physics advances during each query by the method's latency, drawn per query from its distribution measured on one RTX 5090. With paused physics, inference takes no physics time and a policy replans every 50 ms; this control removes the wait for inference, which also raises the feedback rate, and is not an upper bound. Equal latency charges every method the p95 latency of FoveaWAM; original executors let each baseline keep the execution scheme of its paper, such as temporal aggregation for ACT or streaming for DynamicVLA. The latency sweep adds 0 to 200 ms in steps of 50 ms after inference in real time. Rollouts run on RTX PRO 6000 and replay the latency measured on RTX 5090.

Latency measurement. We time each method alone on one RTX 5090 (32 GB), the GPU of our robot, at batch size 1 and in the precision of its release (FP32 for FoveaWAM), over three rounds of 30 warm-up and 300 measured calls on recorded observations with the required causal history. A timed query starts from the newest observation and ends when its first command is available. It includes all computation that needs the newest frame, such as preprocessing, image encoding, the online flow and motion descriptors of PUMA and DynamicWAM, sampling, and action decoding, and on the robot also tracking. The forecast heads of FoveaWAM, skipped at deployment, are timed separately. A

query misses its deadline when this time-to-action exceeds 50 ms, and the action age of a command is its execution time minus the capture time. Appendix C.2 reports the measured values.

B.4. Metrics and Statistics

The task-balanced success rate of method m is

$$\mathrm{SR}(m) = \frac{100}{14} \sum_{k=1}^{14} \frac{1}{n_k} \sum_{i=1}^{n_k} y_{mki}, \tag{S2}$$

with binary outcomes y_{mki} on scene i of task k. The position error at first contact e_p is the distance of the end effector from the expert's contact frame on the target; in the placement tasks, which start with the object held, it uses the contact of the held object with the receptacle and the expert's placement pose. e_p averages over the 13 tasks with a contact, excluding `t3_phase`, and over the episodes that make contact; failures without contact remain in the SR denominator.

Paired intervals. Every method is trained once. Paired 95% intervals bootstrap held-out scenes within each task, jointly across methods, and quantify scene uncertainty for the trained weights; we treat differences whose interval lies within ± 2 points as ties. For the ablations, we also repeat the comparison without the phase-only wire passage (`t3_phase`, Appendix C.4).

B.5. Baselines and Ablation Controls

Table S4. **Baseline training.** All methods train on the same 11,200 demonstrations with the official recipe of each method and run under the shared executor of Appendix B.3. Updates × batch: training budget. H: commands per returned chunk. Params: trainable parameters, frozen encoders excluded. DOM ckpt.: the released DynamicVLA checkpoint trained on DOM. OpenWAM-α generates video and actions jointly with video decoding disabled.

Method	Initialization	Updates×batch	H	Params
ACT [58]	scratch	50,000 × 32	100	[X]
DP [12]	scratch	50,000 × 32	15	[X]
$\pi_{0.5}$ [5]	`pi05_base`	30,000 × 256	10	[X]
$\pi_{0.5}$+SE(3)	`pi05_base`	30,000 × 256	10	[X]
DynamicVLA [46]	DOM ckpt.	50,000 × 44	20	[X]
PUMA [14]	official	[X]×[X]	[X]	[X]
OpenWAM-α [42]	official	34,490 × 192	32	[X]
DynamicWAM [27]	official	40,000 × 128	16	[X]
DynamicWAM+SE(3)	official	40,000 × 128	16	[X]
FoveaWAM	scratch	51,740 × 512	8	523.8M

Common conditions. Each baseline keeps its official architecture, chunk length, initialization, and training recipe (Table S4), trains once on the same 11,200 demonstrations mapped to its action space through the same Cartesian controller, and is configured on development scenes. Rollouts use the agent view each baseline was trained with, any wrist slot masked, and the histories of Table 1; latency is measured in each method's native deployment setting. For the seven configurations timed both with and without a wrist view, the two differ by at most 3.8 ms in p95 latency, under a tenth of the control period; PUMA, OpenWAM with official acceleration, and DynamicWAM+SE(3) were timed only in their deployment setting. DP uses its authors' real-robot DDIM sampler with 16 steps, and OpenWAM is timed with and without its official cache and compilation.

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.1 from the same pose source, with the same validity flags. A two-layer MLP with a zero-initialized output layer maps them to the model's width; for $\pi_{0.5}$ they follow the image tokens in the prefix, where the action expert attends to them, and for DynamicWAM they join its kinematic descriptors. Training follows the base model's recipe.

Ablation controls. The five controls of Sec. 4.3 each change one component of the configuration of Table S2 and keep the data, the recipe with its checkpoint selection, and the executor. *No dedicated SE(3)* removes $\mathcal{L}_P$ and the pose head, so that the target receives only the uniform coarse scene-flow supervision of the whole scene; *no scene-flow supervision* removes $\mathcal{L}_F$ and the flow readout; both keep the target-state tokens and all auxiliary objectives. *No target history* removes target-state slots 0–6, the past poses with their velocity and acceleration features, and keeps slot 7, the current pose (Appendix A.1). *No future supervision* removes $\mathcal{L}_P$, $\mathcal{L}_F$, and the three future appearance losses with their heads, and keeps the current-target reconstruction. *No auxiliary objectives* removes $\mathcal{L}_{\mathrm{aux}}$, the three future appearance losses and the current-target reconstruction, and keeps $\mathcal{L}_P$ and $\mathcal{L}_F$. Since every forecast head is skipped at deployment, only the no-history control changes the inputs at inference.

C. Additional Simulation Results

C.1. Per-Task Results

Table S5 breaks the held-out ConMoto results of Table 1 down by task for every method, with the paired interval of FoveaWAM against the strongest other method on each task; its last two rows give the position error at first contact e_p and the number of episodes with a contact (Appendix B.4). With the real-robot DDIM16 sampler, Diffusion Policy reaches 30.7% SR in real time and 31.0% with physics paused. OpenWAM with cache and compile disabled has no contact in some tasks, so its full 13-task e_p is

Table S5. **Per-task held-out success** (%) on ConMoto for every method of Table 1, real-time protocol. DP: Diffusion Policy. DVLA: DynamicVLA. OWAM: OpenWAM-α with official cache/compile; OW-off: both disabled. DWAM: DynamicWAM. +: with our SE(3) history tokens. σ: FoveaWAM with nominal synthetic correlated pose noise (Appendix C.5). Δ: FoveaWAM minus the strongest other method on that task, with its 95% paired interval. The last two rows give the position error at first contact e_p (mm), averaged over the 13 tasks with a contact, and the number n of episodes with a contact (Appendix B.4); –: no 13-task mean. Task codes are listed in Table S3. Shading of Δ: blue for higher SR, orange for lower, saturating at 15 points. Blue/bold and light blue rank the reported values only; pending entries are excluded.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	OWAM	OW-off	DWAM	DWAM+	σ	**Ours**	Δ [95% CI]
Harmonic grasp	49.0	31.0	51.0	**85.0**	39.0	—	41.0	1.0	50.0	51.0	—	—	—
Conveyor grasp	45.0	45.0	**96.0**	58.0	89.0	—	62.0	0.0	59.0	47.0	—	—	—
Translating grasp	14.0	33.0	59.0	38.0	**61.0**	—	53.0	0.0	53.0	15.0	—	—	—
Rotating grasp	24.0	20.0	32.0	**44.0**	26.0	—	42.0	3.0	30.0	37.0	—	—	—
Occluded grasp	30.0	28.0	37.0	51.0	**80.0**	—	34.0	0.0	44.0	36.0	—	—	—
Moving receptacle	26.0	8.0	26.0	19.0	**39.0**	—	28.0	0.0	0.0	0.0	—	—	—
Rotating receptacle	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	—	**0.0**	**0.0**	**0.0**	**0.0**	—	—	—
Wire passage	99.0	99.0	**100.0**	**100.0**	99.0	—	50.0	58.0	23.0	23.0	—	—	—
Wire passage, full	0.0	3.0	1.0	5.0	**12.0**	—	0.0	0.0	0.0	0.0	—	—	—
Ball capture	36.0	16.0	**46.0**	21.0	41.0	—	45.0	13.0	24.0	23.0	—	—	—
Ball capture, table	71.0	81.0	**100.0**	**100.0**	88.0	—	99.0	88.0	98.0	**100.0**	—	—	—
Slider capture	56.0	30.0	58.0	68.0	28.0	—	**71.0**	31.0	47.0	63.0	—	—	—
Can capture	91.0	31.0	98.0	99.0	72.0	—	**100.0**	5.0	96.0	**100.0**	—	—	—
Post-impact grasp	42.0	5.0	49.0	**51.0**	39.0	—	21.0	22.0	43.0	20.0	—	—	—
Task-balanced	41.6	30.7	**53.8**	52.8	50.9	—	46.1	15.8	40.5	36.8	—	—	—
e_p (mm)	32.2	36.2	31.2	32.5	**30.0**	—	31.7	–	31.2	37.0	—	—	–
n	1110	941	1231	1187	1203	—	1115	439	1193	970	—	—	–

undefined. [Tasks on which every method fails or a baseline outperforms FoveaWAM, and the dominant failure of FoveaWAM.]

C.2. Evaluation Protocols and Latency

Table S6 evaluates every method under three further protocols. Paused physics shows how the relative performance changes when the cost of computation is ignored, equal latency compares all methods at the p95 latency of FoveaWAM, and the original executors check whether the shared executor removes a capability that a baseline relies on. With paused physics, the difference between FoveaWAM and [name] is [$\pm$X.X] points, and under equal latency it is [$\pm$X.X] points. The original executors change the SR of the baselines by at most [X.X] points. Table S7 reports five added-latency points for nine configurations; PUMA and FoveaWAM remain pending. Fig. 4 reads this table directly and plots the selected comparisons; all methods remain in the table.

Table S6. **Protocol controls** on held-out ConMoto scenes (SR, %) under the protocols of Appendix B.3; real time is the protocol of Table 1. Late: queries in real time whose time-to-action exceeds the 50 ms control period (%). Age: p95 action age in real time (ms); –: no command executed. Blue/bold and light blue rank the reported values only; pending entries are excluded.

Method	Paused	Equal latency	Original executor	Real time	Late (%)	Age p95
ACT	40.6	—	—	41.6	0.0	50
Diffusion Policy	31.0	—	—	30.7	100.0	150
$\pi_{0.5}$	66.0	—	—	**53.8**	100.0	150
$\pi_{0.5}$ + SE(3)	**74.9**	—	—	52.8	100.0	150
DynamicVLA	54.6	—	—	50.9	100.0	150
PUMA	—	—	—	—	—	—
OpenWAM-α	58.3	—	—	46.1	100.0	350
OpenWAM-α (off)	58.3	—	—	15.8	100.0	950
DynamicWAM	49.1	—	—	40.5	100.0	250
DynamicWAM + SE(3)	64.1	—	—	36.8	100.0	350
FoveaWAM	—	—	–	—	—	—

Table S7. **Success versus added latency** on 1,400 held-out ConMoto scenes per point (SR in %, real-time protocol, one training seed). The 0 and 150 ms columns equal the Native and +150 ms columns of Table 1. Blue/bold and light blue rank the reported values only; pending entries are excluded.

	Added latency (ms)				
Method	0	50	100	150	200
ACT	41.6	43.5	36.5	24.3	16.5
Diffusion Policy	30.7	26.1	21.0	15.9	14.3
$\pi_{0.5}$	**53.8**	**47.2**	**39.6**	27.8	13.6
$\pi_{0.5}$ + SE(3)	52.8	39.8	32.4	**28.2**	15.1
DynamicVLA	50.9	46.4	30.6	23.4	17.8
PUMA	—	—	—	—	—
OpenWAM-α	46.1	36.1	27.9	24.7	**22.1**
OpenWAM-α (off)	15.8	13.7	14.2	10.4	9.5
DynamicWAM	40.5	27.6	25.0	20.6	20.9
DynamicWAM + SE(3)	36.8	30.6	23.9	21.9	21.3
FoveaWAM	—	—	—	—	—

Timing. Table S8 breaks the time-to-action of FoveaWAM on one RTX 5090 down by stage, measured as in Appendix B.3; Table 1 gives the p95 latency of every method and Table S4 the parameter counts. On the robot, we report the p50, p95, and maximum action age and the fraction of all queries of the test trials that miss the 50 ms deadline, together with the peak memory.

Table S8. **Latency breakdown** of one query of FoveaWAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Image encoding	—	—
World stream (once per query)	—	—
Action stream (5 Euler steps)	—	—
Time-to-action (total)	—	—
Forecast heads (training only, for reference)	—	—
Action age at execution (robot)	—	—
Queries over 50 ms (robot, %)	—	

C.3. Results on DOMINO

On DOMINO [14], we compare FoveaWAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of its clean level-1 setting. All methods use the benchmark's demonstrations, scenes, and native evaluator, one training seed, and 50 episodes per task and method. We report SR and the manipulation score (MS), which scales route completion by penalties for leaving the workspace or view and for collisions (Table 2). FoveaWAM uses domain-specific normalization and reads the simulator pose of the target named by the instruction; slots without a valid pose are masked as in Appendix A.1. The RGB-based baselines do not receive this pose.

Data and adaptation. The baselines share 1,200 training and 240 validation demonstrations, retain all original captures, and execute absolute joint-position targets with DOMINO's native controller. No extra joint-difference filter or policy-side clipping is applied. DynamicWAM retains its official stride-3, horizon-16 sampler and requires at least 10 complete windows per demonstration: 1,054 training and 210 validation sources contribute supervision. The short sources remain in the corpus; two tasks have no eligible training source and one has only four. All 24 tasks remain in the test set. $\pi_{0.5}$ and PUMA use all 1,200/240 sources with their own window rules. All normalizers use training data only.

DynamicWAM adapts the published model's state/action projections to eight dimensions and trains for 40,000 updates at global batch 128; evaluation uses the final checkpoint. $\pi_{0.5}$ starts from its official base and trains for 30,000 updates at global batch 256; evaluation uses its final EMA. PUMA retains the published shared weights, initializes an eight-dimensional output projection, and adapts all originally trainable parameters while keeping its teacher frozen, at global batch 64 on 16 GPUs. Its prespecified budget is 30,000 updates, extendable to 40,000 only with continued validation improvement. Every 2,000 updates, it evaluates all 240 validation sources (18,618 windows), using task-macro normalized action MSE. After at least 10,000 updates, three consecutive checks without a 0.5% relative improvement stop training. Training stopped at 12,000; the lowest validation MSE selected update 6,000. Checkpoint selection and stopping never use test outcomes. The comparison shares data sources and test scenes while retaining method-specific training recipes; it does not match optimization steps or compute across methods.

Results. All three baselines complete the same 1,200 fixed scenes on 32 GPUs, with 25 scenes per arm and task, no missing or duplicate outcomes, and every failure retained. DynamicWAM succeeds in 352 scenes, PUMA in 220, and $\pi_{0.5}$ in 208. DynamicWAM leads on Track (34.9% SR), whereas $\pi_{0.5}$ leads on Intercept (33.8%); PUMA reaches 12.8% and 29.5%, respectively. All averages weight the 24 tasks equally, rather than averaging the two unequal-size task groups. These are single-training-seed results; they do not establish robustness across training seeds. [Add the accepted DOMINO comparison for FoveaWAM.]

C.4. Final Ablations

Table S9 reports the five controls of Sec. 4.3 (Appendix B.5) with native latency and with 150 ms of added latency, each with its paired difference from the full model. The last two columns repeat the paired differences without the phase-only wire passage (`t3_phase`): on development scenes, retraining the same configuration with different seeds changed SR

Table S9. **Final ablations** on held-out ConMoto scenes (Appendix C.4). SR (%) with native latency and with 150 ms of added latency; Δ: control minus FoveaWAM (points) with its paired 95% interval. Last two columns: Δ without the phase-only wire passage (`t3_phase`), 13 tasks. Shading of Δ: blue for higher SR, orange for lower, saturating at 15 points.

	Native		+150 ms		Δ w/o `t3_phase`	
Variant	SR	Δ [95%]	SR	Δ [95%]	Native	+150 ms
FoveaWAM *	—	–	—	–	–	–
No dedicated SE(3) (uniform coarse scene flow)	—	—	—	—	—	—
No scene-flow supervision	—	—	—	—	—	—
No target history	—	—	—	—	—	—
No future supervision	—	—	—	—	—	—
No auxiliary objectives	—	—	—	—	—	—

by up to [7.4] points, mostly on this task, whose success ranged from [0] to [71]% (Appendix B.4). The two task groups of Fig. 5 are listed in Appendix B.1. [Whether any conclusion of Sec. 4.3 changes without `t3_phase`.]

C.5. Robustness to Pose Noise

With frozen weights, we evaluate three synthetic perturbations of the target pose: nominal temporally correlated noise, twice its amplitude, and nominal noise with contiguous missing observations. The nominal process combines a fixed per-episode bias and a stationary correlated component, each with per-axis standard deviations of 5 mm and 1°, with a correlation time of 200 ms. Dropouts last 200 ms, separated by geometrically distributed valid gaps with a mean of 3.8 s, giving a long-run missing fraction of 5%. The perturbation applies to the current and past estimates before the history features are computed.

Fig. S1 evaluates the final model under these perturbations, on the same held-out scenes at zero added latency. Relative to clean poses, SR changes by [X.X], [X.X], and [X.X] points under nominal correlated noise, twice its amplitude, and missing observations.

Figure placeholder: Fig. S1 — pose noise
x: clean poses, nominal correlated noise, twice its amplitude, and nominal noise with missing observations; y: SR of FoveaWAM per task family, frozen weights, real-time protocol; paired intervals over matched scenes.

Figure S1. **Robustness to pose noise.** SR of FoveaWAM under synthetic perturbations of the target pose, relative to clean poses.

C.6. Forecast Diagnostics

Table S10 compares the learned SE(3) forecast of the final model offline with constant-velocity and constant-acceleration extrapolation from the same pose history.

Table S10. **Offline prediction errors.** Target-position error at 400 ms (mm) on the same held-out observations. CV and CA are analytic extrapolations from causal pose history; the learned head reads frozen full-model states.

	Const. vel.	Harmonic	Roll/slide	Impact
CV extrapolation	—	—	—	—
CA extrapolation	—	—	—	—
Learned pose head	—	—	—	—

D. Real-Robot Experiments

D.1. Platform and Tasks

The robot is a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera, commanded through [a timestamped Cartesian interface] at [control frequency]; a fixed third-person camera records RGB-D at [resolution and frame rate], calibrated by [procedure] before each session. FoveaWAM uses this view, baselines whose recipes use a wrist view also receive [wrist camera], and all policies run on one RTX 5090. The task devices are installed on this platform one at a time (Fig. S2; Table S11). The turntable block of T3 has unequal sides and must be grasped across its short sides, so its orientation at contact matters; the toy car of T4 runs at a fixed throttle.

Figure placeholder: Fig. S2 — real-robot platform
Content: one unretouched photo of the shared platform from behind the robot with one task device installed; the third-person RGB-D camera, the robot, and the wrist camera if a baseline uses one, each labeled.

Figure S2. **Real-robot platform** with one task device installed.

Table S11. **Real-robot tasks.** Target: the tracked object. Trial key: what each of the 20 test keys fixes. Speed: target speed measured from the recorded RGB-D of all test trials (m/s, mean ± s.d.). A grasp holds the object between both fingers. Each task has 100 training and 20 validation demonstrations.

Task	Target	Motion; trial key	Speed	Success	Limit (s)
T1	Block	Belt conveyor; [placement on the belt, belt speed]	—	Grasp the block and lift it by [X] cm.	[X]
T2	Open box	Belt conveyor; [box placement, belt speed]; the block starts in the gripper	—	Release the block into the box; it stays inside [until the box leaves the workspace].	[X]
T3	Block, unequal sides	Turntable; [radius, initial angle and yaw of the block, angular speed]	—	Grasp the block across its short sides and lift it by [X] cm.	[X]
T4	Toy car	Remotely driven at a fixed throttle; [start pose, heading]	—	Grasp the car and lift it by [X] cm.	[X]
T5	Tennis ball	[ball-motion mechanism, release position and motion setting]	—	Grasp the tennis ball and lift it by [X] cm.	[X]
T6	Block on cart	Cart translates down an inclined rail; [slope, release point]	—	Grasp the block and lift it by [X] cm.	[X]

Table S12. **Real-robot successes and failures.** Top: successes per task (of 20 trials; tasks in Table S11); All: pooled success rate (%) with its 95% Wilson interval; Δ: paired difference from FoveaWAM (points) with a bootstrap interval over trial keys. Bottom: failures by category over all 120 trials of each method (Appendix D.4); –: not applicable. Shading of Δ: blue for higher SR, orange for lower, saturating at 15 points.

Method	T1	T2	T3	T4	T5	T6	All	Δ
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
ACT	—	—	—	—	—	—	—	—
FoveaWAM	—	—	—	—	—	—	—	–

	Tracking	Late	Misaligned	Slip/drop	Collision	Timeout	Failures
$\pi_{0.5}$	–	—	—	—	—	—	—
DynamicWAM	–	—	—	—	—	—	—
ACT	–	—	—	—	—	—	—
FoveaWAM	—	—	—	—	—	—	—

D.2. Pose Tracking and Labels

Tracking is marker-free and uses one metric mesh per object instance; a ball is defined by its center and radius. In the first frame, Grounding DINO [24] localizes the object named by the task, SAM 2.1 [31] segments it, and FoundationPose [43] registers its pose; afterwards, Cutie [11] propagates the mask and FoundationPose refines a Kalman-filter prediction at each update. An empty mask makes the estimate invalid rather than carrying the previous pose forward, and [re-initialization after a loss]. The tracker updates on the newest frame at every control-period boundary and keeps that frame's capture time, from which the target-state tokens take their finite differences; tracking is part of the measured latency (Table S8).

The target-state inputs of a demonstration replay the same tracker causally over its recorded RGB-D. The pose labels are the tracked poses at future boundaries, [refined offline over the episode], and the flow labels follow [3D point tracks of the recorded RGB-D, as for DROID (Appendix A.3)]. For the ball, the target-state tokens and the pose head use translation only. Table S13 reports how often tracking was lost in the test trials.

Table S13. **Tracking on the real robot.** Per task, over the test trials of FoveaWAM: control-period boundaries, the fraction with an invalid estimate, the longest gap, and re-initializations.

Task	Boundaries	Invalid (%)	Longest gap (s)	Re-init.
T1	—	—	—	—
T2	—	—	—	—
T3	—	—	—	—
T4	—	—	—	—
T5	—	—	—	—
T6	—	—	—	—

D.3. Training and Evaluation Protocol

FoveaWAM is fine-tuned per task from the DROID base (Appendix A.3). $\pi_{0.5}$, DynamicWAM, and ACT [58] train or fine-tune on the same demonstrations with the initialization, recipe, and chunk length of Table S4; DynamicWAM executes [with or without real-time chunking]. Demonstrations, development trials, and test trials come from separate sessions.

Each task has 20 pre-generated trial keys that fix the object instance, its initial pose, and the device setting. For each key, the four methods run consecutively in a random order, so that drift of the apparatus does not align with the methods,

after an operator resets the scene against a reference image. A trial ends at success, failure, a safety stop, or a time limit of about three times the task's median demonstration duration. A key is repeated only if the device deviates from it on its own ([speed outside ±X% of the setting or a device fault]); everything else, including a tracking loss of FoveaWAM, counts as a failure. We report successes per task and the pooled success rate with 95% Wilson intervals, and paired differences from a bootstrap over trial keys within each task. End-to-end time-to-action is logged for every query on the same RTX 5090, including tracking for FoveaWAM.

D.4. Detailed Results and Failures

Table S12 reports the successes per task with intervals and paired differences, and the failures of each method by category. The categories, fixed before the test trials, are observable events: tracking lost (FoveaWAM only), late (the gripper closes behind the target), misaligned grasp, slip or drop after contact, collision or safety stop, and timeout. [Tasks with the largest margin and their failure categories; the most frequent failures of FoveaWAM.]